AI VIET NAM
@aivietnam.edu.vn

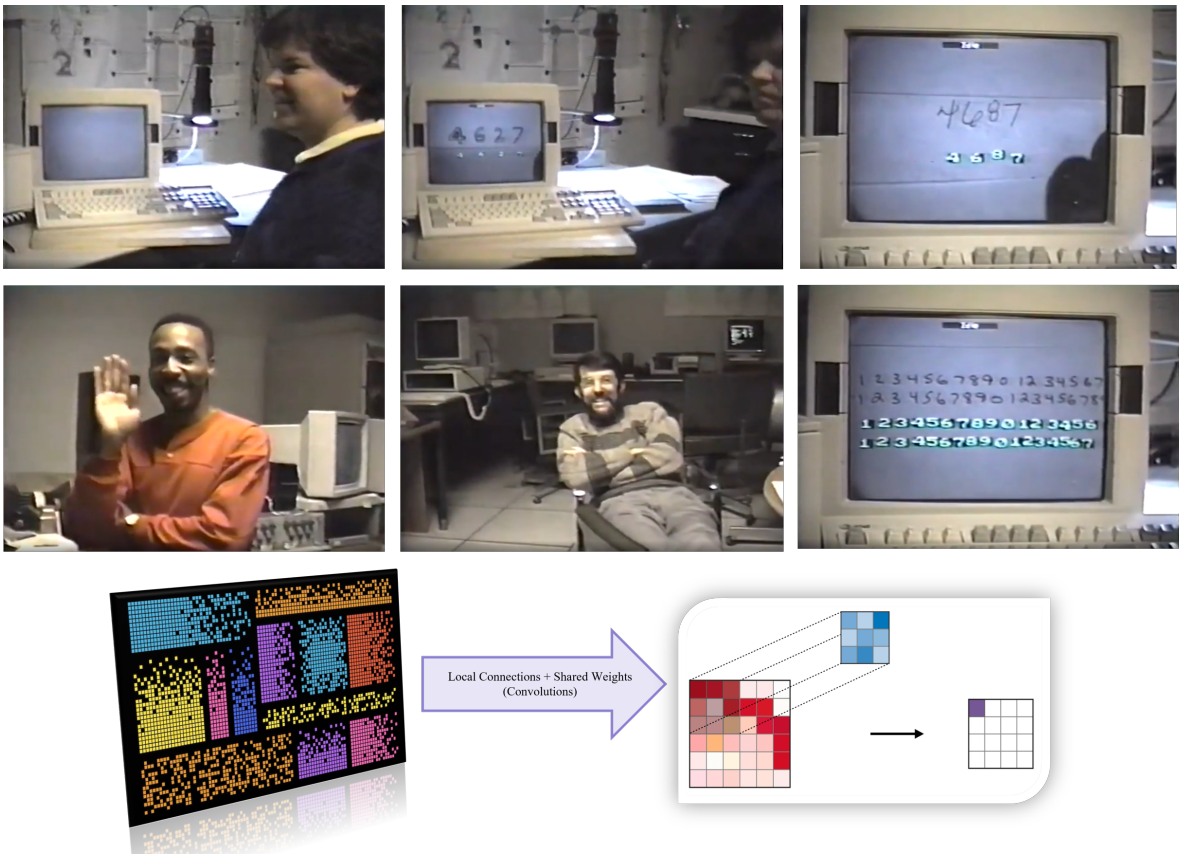
AI VIET NAM – AI COURSE 2025

Exercise: Convolutional Neural Network

Nguyễn Quốc Thái Hồ Quang Hiển Đinh Quang Vinh

I. Giới thiệu

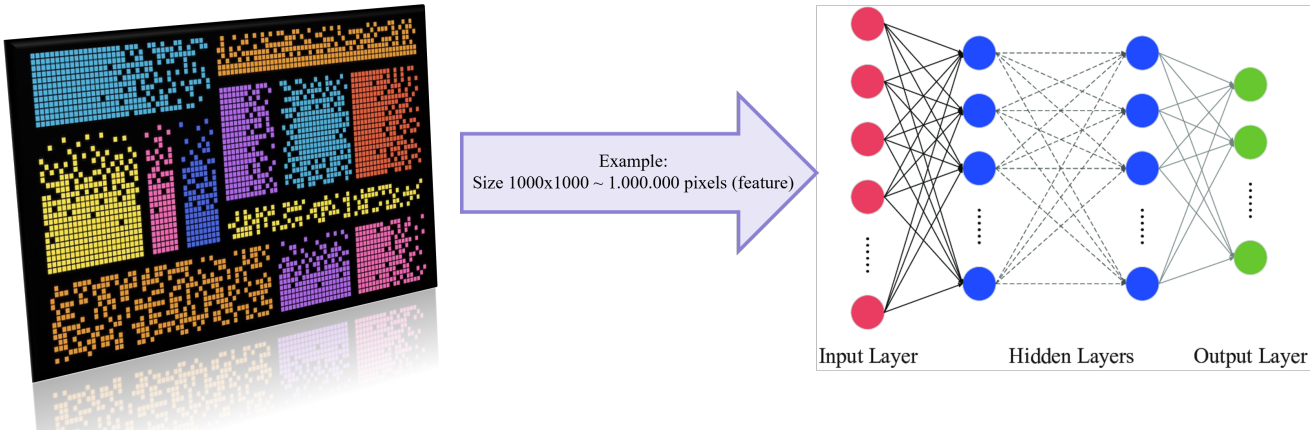
Trước khi Convolutional Neural Network (CNN) ra đời, các bài toán thị giác máy tính chủ yếu dựa vào hai lớp ý tưởng: (i) trích chọn đặc trưng thủ công (hand-crafted features) như edge detector, SIFT, HOG; và (ii) đưa các đặc trưng đó vào những mô hình phân loại nông như MLP, SVM hay k-NN. Khi cố gắng áp dụng MLP fully-connected trực tiếp lên ảnh, số lượng tham số tăng theo kích thước ảnh (ví dụ ảnh 1000×1000 đã cần tới 10^{12} trọng số cho một lớp ẩn), vừa dễ overfit vừa vượt quá tài nguyên tính toán thời bấy giờ. Đồng thời, các mô hình này coi mọi pixel như độc lập, bỏ qua cấu trúc không gian rất rõ ràng của ảnh.



Hình 1: Yann LeCun và các cộng sự giới thiệu CNN.

Từ góc nhìn sinh học, các nhà nghiên cứu cũng biết rằng vỏ não thị giác của động vật tổ chức theo các vùng tiếp nhận cục bộ (receptive field) và chia sẻ kiểu phản ứng giống nhau trên toàn bộ trường nhìn. Những quan sát này dẫn tới các mô hình như Neocognitron của Fukushima và sau đó là các CNN hiện đại, nơi mà kết nối cục bộ và chia sẻ trọng số trở thành hai nguyên lý cốt lõi.

Bước ngoặt lớn đến vào cuối thập niên 1980–1990 khi Yann LeCun và cộng sự đề xuất LeNet: một mạng nhiều tầng gồm các lớp convolution, pooling và fully-connected, được huấn luyện end-to-end bằng backpropagation để nhận dạng chữ viết tay và sau này là bộ dữ liệu MNIST. Thay vì phải thiết kế đặc trưng bằng tay, CNN học trực tiếp các bộ lọc từ dữ liệu, xây dựng dần dần các biểu diễn phân cấp từ cạnh, góc, texture cho tới cấu trúc đối tượng, với số tham số ít hơn rất nhiều so với MLP thuần túy. Trong buổi học hôm nay, chúng ta sẽ quay lại những ý tưởng nền tảng đó thông qua các bài tập về CNN cơ bản.



Hình 2: Sự hạn chế khi áp dụng MLP trên ảnh có kích thước lớn.

Mục lục

I. Giới thiệu **1**

II. Vì sao cần mạng CNN? **4**

II.1. Vấn đề khi dùng MLP trực tiếp trên ảnh 4

II.2. Hai tính chất quan trọng của ảnh 4

II.3. Ý tưởng cốt lõi của CNN 4

II.4. Locally connected layer 5

II.5. Shared parameters (weight sharing) 5

III. Các khái niệm chính trong CNN **6**

III.1. Convolution Layer 6

III.2. Pooling Layer 7

III.3. Fully-Connected Layer 8

IV. Thực hành xây dựng mô hình CNN **10**

IV.1. Mô hình phân loại chữ số trên tập dữ liệu MNIST bằng LeNet Model 10

IV.2. Mô hình phân loại trên tập dữ liệu Cassava Leaf Disease 17

IV.3. Phân loại văn bản bằng mô hình TextCNN 21

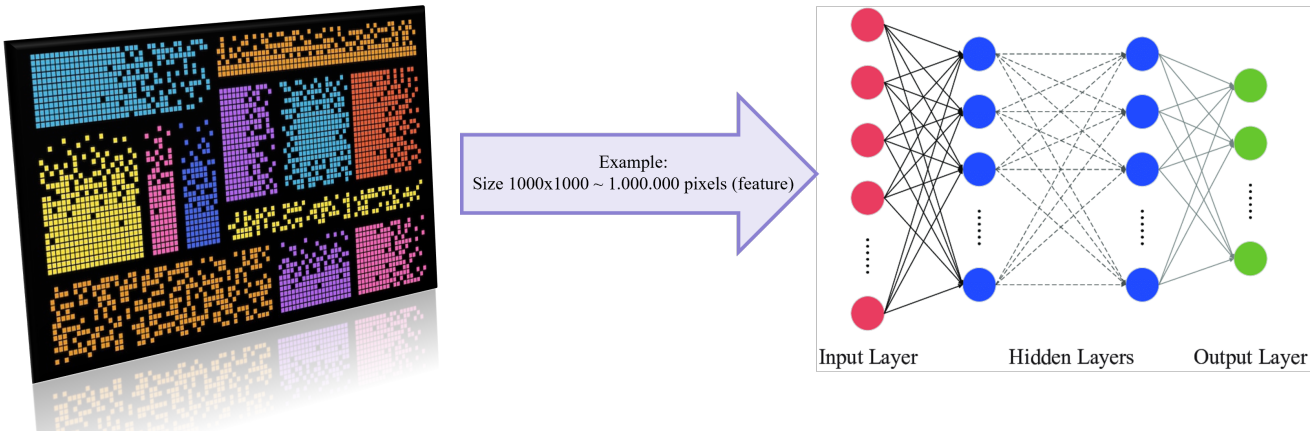
V. Câu hỏi trắc nghiệm **29**

Phụ lục **32**

II. Vì sao cần mạng CNN?

II.1. Vấn đề khi dùng MLP trực tiếp trên ảnh

Nếu xem mỗi pixel của ảnh là một feature và đưa toàn bộ ảnh vào một mạng nơ-ron truyền thẳng (MLP) fully-connected, số lượng tham số sẽ tăng rất nhanh. Chẳng hạn, một ảnh kích thước 1000×1000 có 1,000,000 pixels. Nếu một lớp ẩn có 1,000,000 nút và mỗi nút kết nối với toàn bộ ảnh, ta sẽ có khoảng 10^{12} trọng số chỉ riêng cho lớp đó. Một mô hình với số lượng tham số khổng lồ như vậy rất khó huấn luyện ổn định, cực kỳ dễ bị overfit, đòi hỏi lượng dữ liệu huấn luyện rất lớn, tốn nhiều bộ nhớ và tài nguyên tính toán, và trong thực tế là không khả thi cho các bài toán ảnh kích thước lớn.



Hình 3: Sự hạn chế khi áp dụng MLP trên ảnh có kích thước lớn.

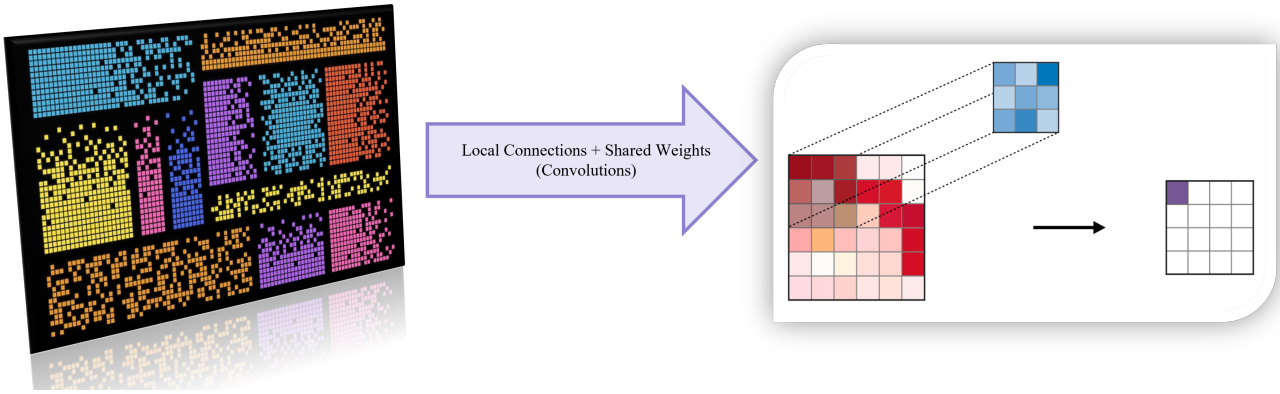
II.2. Hai tính chất quan trọng của ảnh

Thay vì coi mỗi pixel là độc lập, ta có thể khai thác cấu trúc của ảnh. Thứ nhất, **feature localization**: mỗi pixel hoặc feature thường chỉ liên quan mạnh đến các pixel lân cận, tức là thông tin quan trọng nằm trong các vùng cục bộ nhỏ (local neighborhoods). Thứ hai, **feature independence of location**: cùng một kiểu pattern (ví dụ một cạnh dọc, một góc vuông, một nét cong) có ý nghĩa giống nhau dù xuất hiện ở góc trên hay dưới của bức ảnh. Nói cách khác, ta quan tâm *có* feature đó hay không hơn là *nó nằm ở đâu một cách tuyệt đối.*

II.3. Ý tưởng cốt lõi của CNN

Convolutional Neural Network được thiết kế để khai thác trực tiếp hai tính chất trên. Thay vì kết nối đầy đủ từ mọi pixel tới mọi neuron như MLP, CNN sử dụng các *locally connected networks* (mỗi neuron chỉ nhìn một vùng nhỏ của ảnh) kết hợp với *shared parameters* (cùng một bộ trọng số được dùng cho nhiều vị trí khác nhau). Sự kết hợp này tạo thành các **convolutional layers**, vừa giảm mạnh số tham số, vừa giữ được khả năng phát hiện cùng một feature ở nhiều vị trí

trên ảnh. Nhờ đó, CNN trở thành kiến trúc tự nhiên và hiệu quả cho các bài toán nhận dạng, phân loại và xử lý ảnh.



Hình 4: Minh họa cách áp dụng CNN cho ảnh.

II.4. Locally connected layer

Trong một **locally connected layer**, mỗi neuron không còn nhận toàn bộ ảnh làm đầu vào mà chỉ nhận một "cửa sổ" nhỏ (small portion) xung quanh một vị trí nhất định, gọi là *local receptive field*. Ví dụ, thay vì nhìn toàn bộ ảnh 1000×1000, một neuron chỉ nhìn một vùng 5×5 hoặc 7×7. Nhờ chỉ kết nối tới một phần nhỏ của ảnh, số kết nối (và do đó số tham số) giữa input và hidden layer giảm đi rất nhiều, trong khi vẫn phù hợp với tính chất local của ảnh.

II.5. Shared parameters (weight sharing)

Không chỉ giới hạn kết nối trong vùng local, CNN còn áp dụng **shared parameters**: cùng một bộ trọng số (filter/kernel) được trượt qua nhiều vị trí khác nhau trong ảnh. Một filter có thể được hiểu như một "bộ dò feature" (ví dụ cạnh dọc, cạnh ngang, texture), và ta dùng đúng bộ trọng số đó để quét toàn bộ ảnh. Điều này phản ánh giả thuyết rằng cách phát hiện feature đó là như nhau ở mọi vị trí. Việc chia sẻ trọng số vừa giúp mô hình học được các pattern ổn định theo vị trí (translation invariance), vừa tiếp tục giảm số lượng tham số cần học, làm cho CNN hiệu quả hơn rất nhiều so với MLP truyền thống trên dữ liệu ảnh.

III. Các khái niệm chính trong CNN

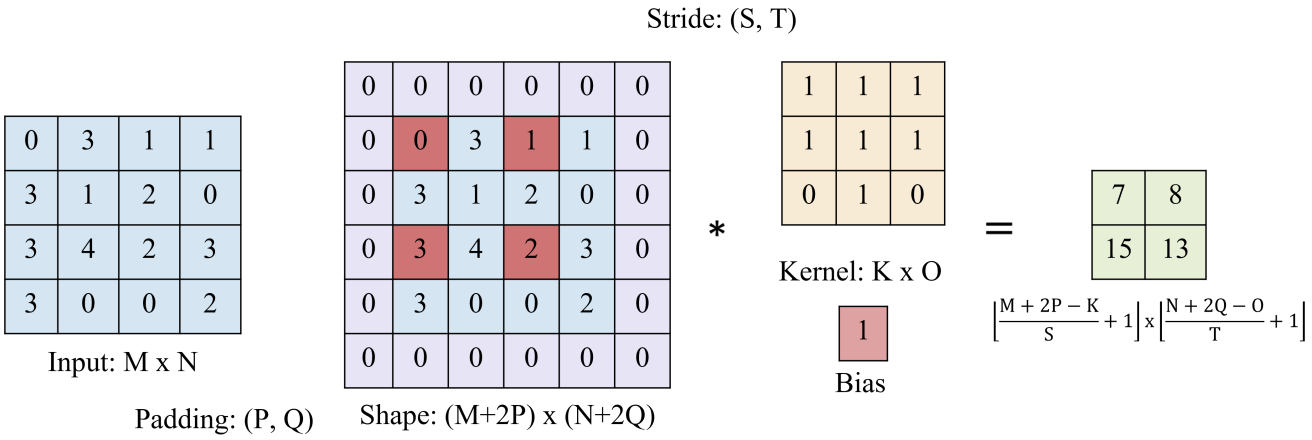
III.1. Convolution Layer

Convolution layer là lớp cốt lõi của CNN, chịu trách nhiệm trích xuất *đặc trưng cục bộ* từ ảnh. Thay vì kết nối mỗi neuron với toàn bộ ảnh, mỗi neuron chỉ nhìn một vùng nhỏ gọi là *local receptive field*. Việc "quét" cùng một bộ trọng số trên toàn ảnh (weight sharing) tạo ra các *feature maps* thể hiện mức độ xuất hiện của một pattern (cạnh, góc, texture, ...) tại từng vị trí.

Filter/Kernel: Filter (hay kernel) là một ma trận trọng số nhỏ, ví dụ 3×3 hoặc 5×5. Mỗi filter được áp dụng (convolve) lên từng vị trí của ảnh: tại mỗi bước, ta nhân từng phần tử của filter với vùng ảnh tương ứng rồi cộng lại, cho ra một giá trị trên feature map. Nhiều filter khác nhau sẽ học được nhiều loại đặc trưng khác nhau (cạnh ngang, cạnh dọc, chấm tròn, ...). Về mặt toán học, với ảnh đầu vào kích thước (C_{in}, H, W) và C_{out} filter kích thước $(C_{\text{in}}, K_h, K_w)$, ta thu được đầu ra kích thước $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$.

Padding: Padding là thao tác thêm các pixel "giả" (thường là 0) xung quanh biên ảnh. Mục đích là kiểm soát kích thước đầu ra và tránh mất thông tin ở rìa ảnh. Nếu không padding (valid convolution), mỗi lần apply kernel sẽ làm giảm kích thước ảnh. Nếu chọn padding phù hợp ("same" convolution), ta có thể giữ nguyên chiều cao, chiều rộng sau convolution.

Stride: Stride là bước nhảy của filter khi trượt trên ảnh. Stride càng lớn thì số vị trí tính toán càng ít, đầu ra càng nhỏ. Nếu stride bằng 1, filter di chuyển từng pixel một; nếu stride bằng 2, filter "nhảy cóc" 2 pixel mỗi lần. Stride cho phép ta vừa trích xuất đặc trưng vừa giảm dần độ phân giải không gian.



Hình 5: Mô phỏng đơn giản về Convolution Layer.

Kích thước đầu ra: Với ảnh đầu vào kích thước (H, W), kernel (K_h, K_w), padding (P_h, P_w), stride (S_h, S_w), kích thước đầu ra được tính:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Ví dụ ngắn: Ảnh 28×28, kernel 3×3, padding 0, stride 1:

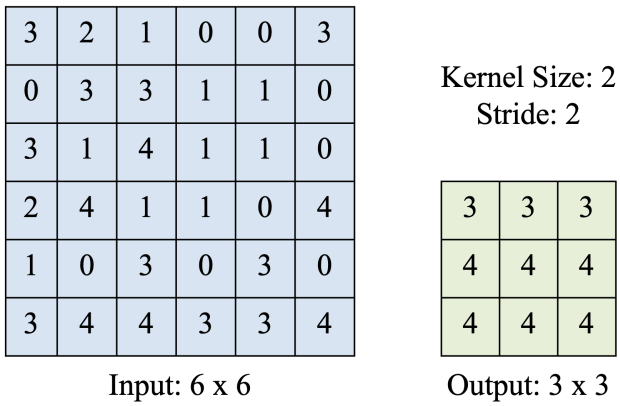
$$H_{\text{out}} = W_{\text{out}} = \left\lfloor \frac{28-3}{1} \right\rfloor + 1 = 26.$$

Nếu dùng 6 filter, đầu ra có shape $6 \times 26 \times 26$.

III.2. Pooling Layer

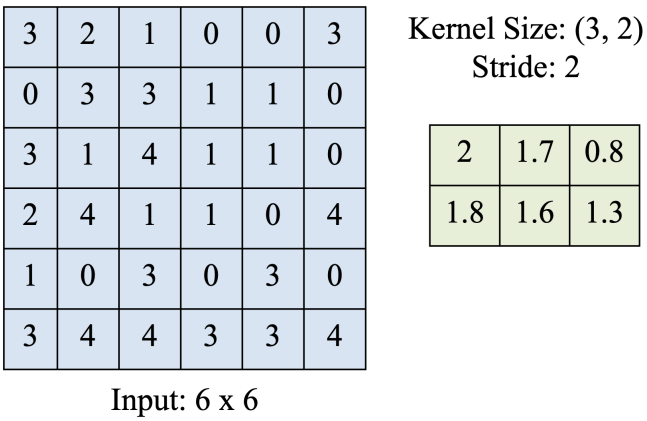
Pooling layer dùng để *giảm kích thước không gian* của feature map và tăng tính bất biến với những dịch chuyển nhỏ, biến dạng nhẹ trong ảnh. Thay vì giữ mọi giá trị trên feature map, pooling tóm tắt thông tin trong từng vùng nhỏ bằng một số đại diện.

Max-Pooling: Max-pooling chọn *giá trị lớn nhất* trong mỗi cửa sổ pooling (ví dụ 2×2). Ý tưởng là giá trị lớn nhất thể hiện mức kích hoạt mạnh nhất của feature trong vùng đó. Điều này vừa giảm kích thước, vừa giữ lại các tín hiệu mạnh nhất, giúp mô hình ít nhạy cảm với việc feature dịch chuyển vài pixel.



Hình 6: Mô phỏng đơn giản về Max Pooling.

Average-Pooling: Average-pooling lấy *giá trị trung bình* trong cửa sổ pooling. Cách này ít "hung hăng" hơn max-pooling, phù hợp khi ta muốn giữ thông tin dưới dạng trung bình vùng.



Hình 7: Mô phỏng đơn giản về Average Pooling.

Kích thước đầu ra: Với đầu vào kích thước (C, H, W), kernel pooling (K_h, K_w), stride (S_h, S_w) (không padding), ta có:

$$H_{\text{out}} = \left\lfloor \frac{H - K_h}{S_h} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W - K_w}{S_w} \right\rfloor + 1,$$

và số kênh C được giữ nguyên.

Ví dụ ngắn: Feature map $6 \times 26 \times 26$, max-pooling 2×2 với stride 2:

$$H_{\text{out}} = W_{\text{out}} = \left\lfloor \frac{26 - 2}{2} \right\rfloor + 1 = 13,$$

nên đầu ra là $6 \times 13 \times 13$.

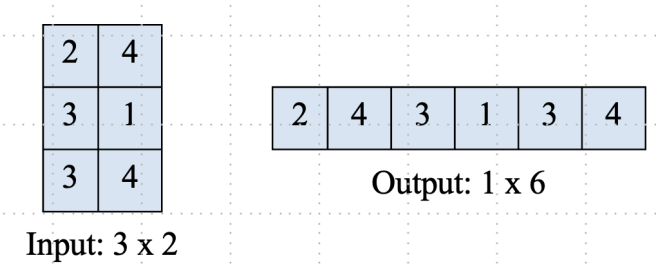
III.3. Fully-Connected Layer

Sau nhiều lớp convolution và pooling, ta thu được một tensor có kích thước $(C_{\text{final}}, H_{\text{final}}, W_{\text{final}})$ chứa các đặc trưng trừu tượng ở mức cao. Để thực hiện phân loại, tensor này được *flatten* thành một vector 1 chiều và đưa vào các *fully-connected layers* (dense layers), tương tự như trong MLP.

Flatten: Flatten biến tensor $(C_{\text{final}}, H_{\text{final}}, W_{\text{final}})$ thành một vector duy nhất có độ dài:

$$D = C_{\text{final}} \cdot H_{\text{final}} \cdot W_{\text{final}}.$$

Vector này đóng vai trò như "đặc trưng toàn cục" (global feature vector) của ảnh sau khi đã qua nhiều tầng trích xuất đặc trưng.



Hình 8: Mô phỏng đơn giản về Flatten.

Fully-Connected Layers: Các fully-connected layers ánh xạ vector đặc trưng D-chiều sang không gian nhãn. Ví dụ, với bài toán phân loại K lớp, lớp cuối cùng thường là một layer Linear $D' \to K$ kèm theo softmax để ra phân phối xác suất trên các lớp. Các lớp fully-connected cho phép kết hợp các đặc trưng đã học ở nhiều kênh và nhiều vị trí khác nhau thành quyết định cuối cùng.

Ví dụ ngắn: Nếu sau backbone CNN ta có đầu ra $(16, 4, 4)$, thì:

$$D = 16 \cdot 4 \cdot 4 = 256.$$

Ta có thể dùng một kiến trúc:

$$\mathrm{FC}_1 : \mathbb{R}^{256} \to \mathbb{R}^{120}, \quad \mathrm{FC}_2 : \mathbb{R}^{120} \to \mathbb{R}^{10},$$

với 10 là số lớp cần phân loại (ví dụ 10 chữ số trong MNIST).

IV. Thực hành xây dựng mô hình CNN

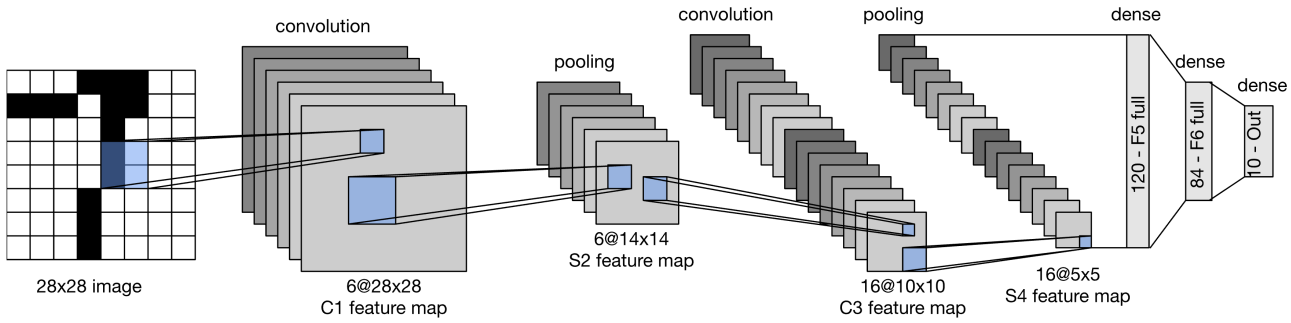
IV.1. Mô hình phân loại chữ số trên tập dữ liệu MNIST bằng LeNet Model

LeNet là một trong những kiến trúc CNN kinh điển cho bài toán nhận dạng chữ viết tay. Mô hình được thiết kế để làm việc với ảnh xám kích thước 28×28 (như bộ dữ liệu MNIST), sau đó trích xuất dần các đặc trưng không gian thông qua các lớp Convolution và Pooling trước khi đưa vào các lớp Fully-Connected để phân loại 10 chữ số từ 0 đến 9.

Kiến trúc LeNet mà chúng ta sử dụng trong bài này gồm chuỗi các tầng:

- **C1:** Convolution với 6 filter, cho ra $6@28 \times 28$ feature maps.
- **S2:** Pooling (thường là AvgPool hoặc MaxPool) giảm xuống $6@14 \times 14$.
- **C3:** Convolution với 16 filter, cho ra $16@10 \times 10$ feature maps.
- **S4:** Pooling tiếp tục giảm còn $16@5 \times 5$.
- **Flatten:** Duỗi tensor $16 \times 5 \times 5$ thành vector 400 chiều.
- **F5, F6, Out:** Ba lớp Fully-Connected lần lượt có 120, 84 và 10 neuron để dự đoán xác suất thuộc 10 lớp số.

Trong các phần tiếp theo, chúng ta sẽ hiện thực hoá kiến trúc này bằng `PyTorch`: xây dựng class `LeNetClassifier`, huấn luyện trên tập MNIST, đánh giá độ chính xác trên tập test và trực quan hoá các feature maps mà LeNet học được ở những lớp convolution đầu tiên.



Hình 9: Mô phỏng đơn giản về mô hình LeNet.

Bước 1: Import thư viện và tải bộ dữ liệu MNIST

```python
import os
import random
import numpy as np
import time

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.utils.data as data

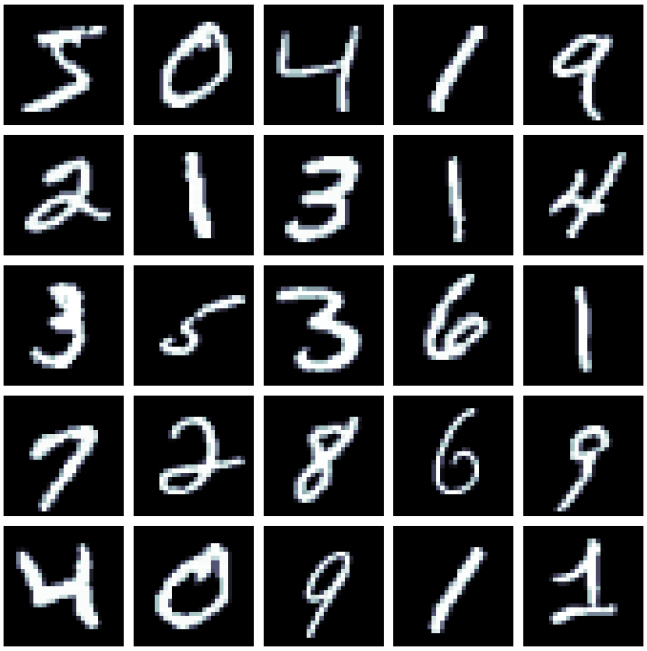
import torchvision.transforms as transforms
import torchvision.datasets as datasets

from torchsummary import summary
import matplotlib.pyplot as plt
from PIL import Image

ROOT = "./data"

train_data = datasets.MNIST(root=ROOT, train=True, download=True)
test_data  = datasets.MNIST(root=ROOT, train=False, download=True)
```

Ở bước này, ta import đầy đủ các thư viện cần thiết cho việc xây dựng và huấn luyện CNN (PyTorch, Torchvision, Matplotlib, PIL, . . .), đồng thời khai báo đường dẫn lưu dữ liệu thông qua biến `ROOT`. Hai lệnh `datasets.MNIST` sẽ tự động tải (nếu chưa có) và tạo ra hai tập `train_data` và `test_data`, tương ứng với tập huấn luyện (60 000 ảnh) và tập kiểm tra (10 000 ảnh) của bộ dữ liệu MNIST kích thước 28×28 pixels dùng cho những bước tiếp theo.



Hình 10: Tập dữ liệu phân loại chữ số MNIST.

Bước 2: Chia tập train/validation, chuẩn hóa và tạo DataLoader

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_data) * VALID_RATIO)
n_valid_examples = len(train_data) - n_train_examples

train_data, valid_data = data.random_split(train_data,
                                           [n_train_examples, n_valid_examples])

# Compute mean and std for normalization
mean = train_data.dataset.data.float().mean() / 255
std  = train_data.dataset.data.float().std() / 255

# Define transforms
train_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean], std=[std])
])

test_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean], std=[std])
])

train_data.dataset.transform = train_transforms
valid_data.dataset.transform = test_transforms
test_data.transform          = test_transforms
```

```
26
27 BATCH_SIZE = 256
28
29 train_dataloader = data.DataLoader(train_data,
30                                     shuffle=True,
31                                     batch_size=BATCH_SIZE)
32 valid_dataloader = data.DataLoader(valid_data,
33                                     batch_size=BATCH_SIZE)
34 test_dataloader  = data.DataLoader(test_data,
35                                     batch_size=BATCH_SIZE)
```

Ở bước này, ta chia tập `train_data` ban đầu thành hai phần: tập huấn luyện và tập validation. Biến `VALID_RATIO = 0.9` nghĩa là dùng 90% mẫu cho huấn luyện và 10% còn lại cho validation, được thực hiện bằng hàm `data.random_split`.

Tiếp theo, ta ước lượng **mean** và **std** của ảnh MNIST dựa trên dữ liệu huấn luyện để chuẩn hóa: mỗi pixel được đưa về khoảng xấp xỉ phân phối chuẩn thông qua phép biến đổi `Normalize(mean=[mean], std=[std])`. Các phép biến đổi này được gán cho tập train, validation và test để đảm bảo dữ liệu ở mọi giai đoạn đều được chuẩn hóa giống nhau.

Cuối cùng, ta tạo các `DataLoader` cho train/validation/test với `batch_size = 256`. Tập train được bật `shuffle=True` để trộn thứ tự mẫu ở mỗi epoch, giúp quá trình học ổn định hơn; trong khi tập validation và test giữ nguyên thứ tự (không shuffle) để đánh giá mô hình một cách nhất quán.

Bước 3: Khai báo kiến trúc LeNet

Code Exercise

```
1  class LeNetClassifier(nn.Module):
2      def __init__(self, num_classes, in_channels=1, img_size=28):
3          super().__init__()
4          self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=6,
5                                 kernel_size=5, padding="same")
6          self.avgpool1 = nn.AvgPool2d(kernel_size=2)
7          self.conv2 = nn.Conv2d(***Your code here***)
8          self.avgpool2 = nn.AvgPool2d(***Your code here***)
9          self.flatten = nn.Flatten()
10
11         # Calculate fc input size dynamically
12         if img_size == 28:
13             fc_input = 16 * 5 * 5
14         else:  # for 150x150 images
15             fc_input = 16 * 35 * 35
16
17         self.fc_1 = nn.Linear(fc_input, 120)
18         self.fc_2 = nn.Linear(120, 84)
19         self.fc_3 = nn.Linear(***Your code here***)
20
```

```
    def forward(self, inputs):
        outputs = self.conv1(inputs)
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = ***Your code here***
        outputs = self.fc_3(outputs)
        return outputs
```

Trong bước này, bạn cần **hoàn thiện kiến trúc LeNet** bằng cách điền vào các vị trí `***Your code here***` sao cho mô hình khớp với kiến trúc CNN cơ bản đã mô tả ở phần trước.

Yêu cầu phần ___init___:

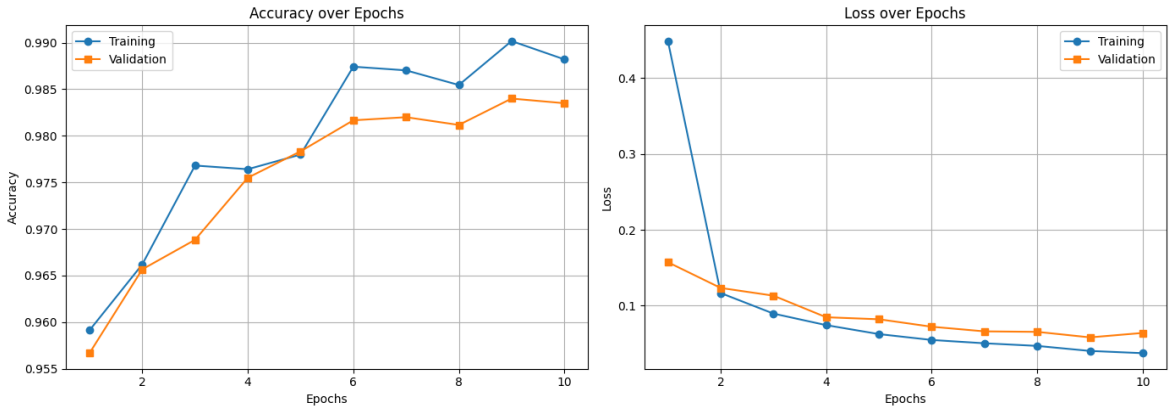
- Hoàn thiện lớp `conv2`: nhận đầu vào là các feature map do `conv1` sinh ra, tạo ra số lượng feature map lớn hơn ở tầng tiếp theo và sử dụng kernel 2D kích thước 5×5 không có padding.
- Hoàn thiện lớp `avgpool2`: sử dụng một lớp pooling 2D với cửa sổ và stride giống `avgpool1` để tiếp tục giảm kích thước không gian của feature map sau `conv2`.
- Hoàn thiện lớp `fc_3`: kết nối đầu ra từ lớp `fc_2` tới một lớp fully-connected có số chiều đầu ra đúng bằng `num_classes` để mô hình dự đoán xác suất cho mỗi lớp.

Yêu cầu phần `forward`:

- Sau khi đưa `inputs` qua `conv1`, áp dụng lớp pooling thứ nhất `avgpool1` và một hàm kích hoạt phi tuyến (ví dụ ReLU) lên kết quả.
- Tiếp theo, truyền tensor qua `conv2`, sau đó áp dụng lớp pooling thứ hai `avgpool2` và tiếp tục áp dụng hàm kích hoạt phi tuyến.
- Sau hai khối Convolution + Pooling, sử dụng `self.flatten` để duỗi tensor 3D thành vector 1D rồi lần lượt đưa qua `fc_1` và `fc_2` trước khi đi vào `fc_3` để tạo ra logits cho các lớp.

Bước 4: Huấn luyện mô hình và visualize kết quả

Chi tiết đoạn code huấn luyện (vòng lặp epoch, lựa chọn loss function, optimizer, logging, lưu mô hình, ...) được trình bày đầy đủ ở phần Phụ lục. Trong phần nội dung chính này, chúng ta chỉ tóm tắt lại quá trình và kết quả huấn luyện để tránh làm bài viết quá dài.



Hình 11: Accuracy và loss của quá trình huấn luyện model.

Dựa trên log huấn luyện, có thể quan sát một số điểm chính:

- Ngay từ epoch đầu tiên, accuracy đã tăng nhanh và đạt khoảng 0.96 trên tập train, 0.96 trên tập validation.
- Sau vài epoch, mô hình ổn định quanh Train Acc ≈ 0.99 và Valid Acc ≈ 0.98, Train/Valid Loss đều giảm và không có dấu hiệu overfitting rõ rệt.
- Thời gian mỗi epoch khoảng 12–14 giây, phù hợp với một CNN nhỏ trên ảnh xám 28×28.

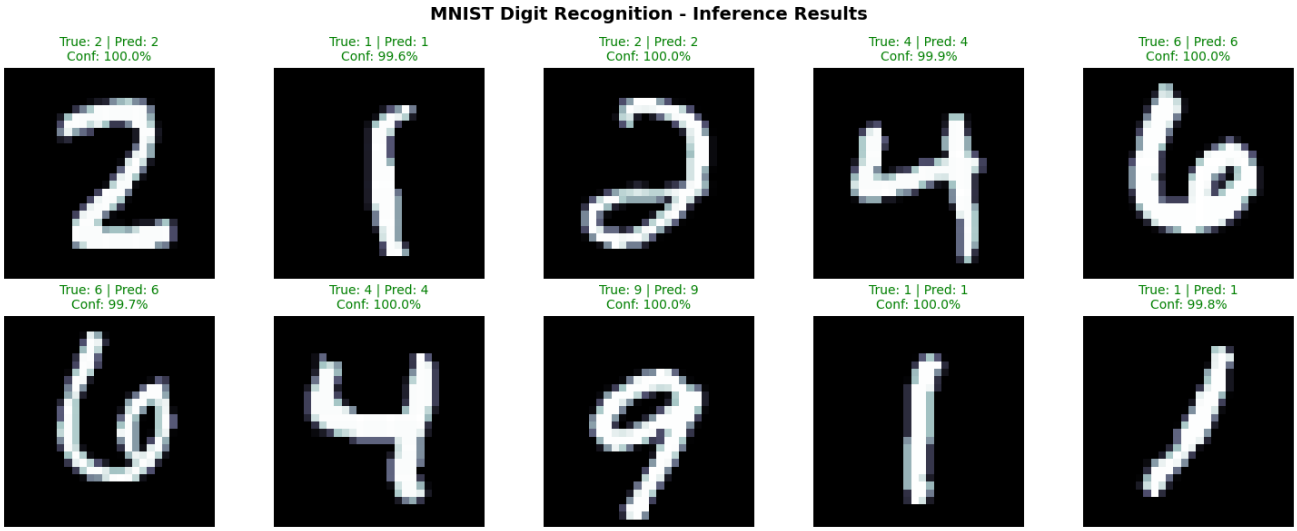


Hình 12: Trực quan hóa các filter của tầng Conv1.

Sau khi huấn luyện xong, ta sử dụng một hàm trợ giúp `plot_filtered_images` để quan sát trực tiếp những gì tầng `conv1` đã học được. Ý tưởng là:

- Lấy một vài ảnh từ tập test (ví dụ 5 ảnh đầu tiên) và lấy các trọng số filter của `lenet_model.conv1`.
- Áp dụng phép tích chập (`F.conv2d`) giữa các ảnh này và từng filter để thu được các feature map tương ứng.
- Vẽ mỗi ảnh gốc cùng với các phiên bản đã được filter ở cạnh bên, giúp ta thấy mỗi filter làm nổi bật kiểu đặc trưng nào (cạnh, nét cong, độ tương phản, . . .).

Nhờ trực quan hóa này, người học có thể hiểu rõ hơn vai trò của các lớp convolution: mỗi filter hoạt động như một bộ dò đặc trưng, còn feature map chính là hình ảnh biểu diễn mức độ kích hoạt của đặc trưng đó trên toàn bộ ảnh đầu vào.



Hình 13: Kết quả inference model trên tập test.

Sau khi mô hình LeNet được huấn luyện xong, ta tiến hành bước suy luận (inference) để kiểm tra khả năng nhận dạng chữ số trên các ảnh chưa từng thấy trong quá trình train. Một hàm tiện ích được sử dụng để nhận vào một ảnh MNIST (dưới dạng tensor), đưa qua mô hình ở chế độ *evaluation* và trả về nhãn dự đoán cùng với độ tự tin (confidence) dưới dạng phần trăm.

Để minh họa, ta chọn ngẫu nhiên một số ảnh từ tập test (ví dụ 10 ảnh), chạy inference cho từng ảnh và trực quan hóa kết quả trên một lưới gồm 2 hàng, 5 cột. Mỗi ô hiển thị ảnh chữ số gốc kèm tiêu đề chứa nhãn thật, nhãn dự đoán và độ tự tin của mô hình. Tiêu đề được tô màu xanh lá nếu mô hình dự đoán đúng và màu đỏ nếu dự đoán sai, giúp người quan sát nhanh chóng đánh giá trực quan hiệu quả nhận dạng của LeNet trên tập dữ liệu MNIST.

IV.2. Mô hình phân loại trên tập dữ liệu Cassava Leaf Disease

Bước 1: Chuẩn bị và tiền xử lý dữ liệu

```
!wget --no-check-certificate https://storage.googleapis.com/emcassavadata/
                                        cassavaleafdata.zip -O ./cassavaleafdata.
                                        zip
!unzip ./cassavaleafdata.zip -d ./cassavaleafdata/

labels_dict = {
    "cbb": "Cassava Bacterial Blight (CBB)",
    "cbsd": "Cassava Brown Streak Disease (CBSD)",
    "cgm": "Cassava Green Mottle (CGM)",
    "cmd": "Cassava Mosaic Disease (CMD)",
    "healthy": "Healthy"
}

img_size = 150

train_transforms_cassava = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.ToTensor(),
])

# Load datasets
cassava_train_data = datasets.ImageFolder(root=data_paths["train"],
                                          transform=train_transforms_cassava)
cassava_valid_data = datasets.ImageFolder(root=data_paths["valid"],
                                          transform=train_transforms_cassava)
cassava_test_data  = datasets.ImageFolder(root=data_paths["test"],
                                          transform=train_transforms_cassava)

BATCH_SIZE_CASSAVA = 256

cassava_train_dataloader = data.DataLoader(cassava_train_data,
                                           shuffle=True,
                                           batch_size=BATCH_SIZE_CASSAVA)
cassava_valid_dataloader = data.DataLoader(cassava_valid_data,
                                           batch_size=BATCH_SIZE_CASSAVA)
cassava_test_dataloader  = data.DataLoader(cassava_test_data,
                                           batch_size=BATCH_SIZE_CASSAVA)
```

Ở bước này, ta tải tập dữ liệu lá sắn từ nguồn lưu trữ trên Google Cloud, giải nén vào thư mục làm việc và tổ chức lại dữ liệu theo ba phần: `train`, `valid` và `test`. Từ điển `labels_dict` dùng để ánh xạ các nhãn viết tắt (`"cbb"`, `"cbsd"`, `"cgm"`, `"cmd"`, `"healthy"`) sang tên đầy đủ của 5 loại bệnh (hoặc trạng thái khỏe mạnh) trên lá sắn.

Mỗi ảnh được resize về kích thước cố định 150×150 và chuyển thành tensor thông qua `transforms.Compose`. Sau đó, `datasets.ImageFolder` tự động đọc ảnh theo cấu trúc thư mục

để tạo ra các tập `cassava_train_data`, `cassava_valid_data`, `cassava_test_data`. Cuối cùng, các `DataLoader` tương ứng được khởi tạo với `batch_size = 256`; trong đó tập train được bật `shuffle=True` để trộn dữ liệu ở mỗi epoch, còn tập validation và test giữ nguyên thứ tự nhằm phục vụ đánh giá mô hình.

Bước 2: Khởi tạo và huấn luyện LeNet cho bài toán lá sắn

```python
# Model setup
num_classes_cassava = len(cassava_train_data.classes)
cassava_model = LeNetClassifier(num_classes_cassava,
                                in_channels=3,
                                img_size=150).to(device)

criterion_cassava = nn.CrossEntropyLoss()
optimizer_cassava = optim.Adam(cassava_model.parameters(), lr=2e-4)

# Training loop
num_epochs = 10
save_model = "./model"
os.makedirs(save_model, exist_ok=True)

train_accs, train_losses = [], []
eval_accs, eval_losses = [], []
best_loss_eval = 100

for epoch in range(1, num_epochs + 1):
    epoch_start_time = time.time()

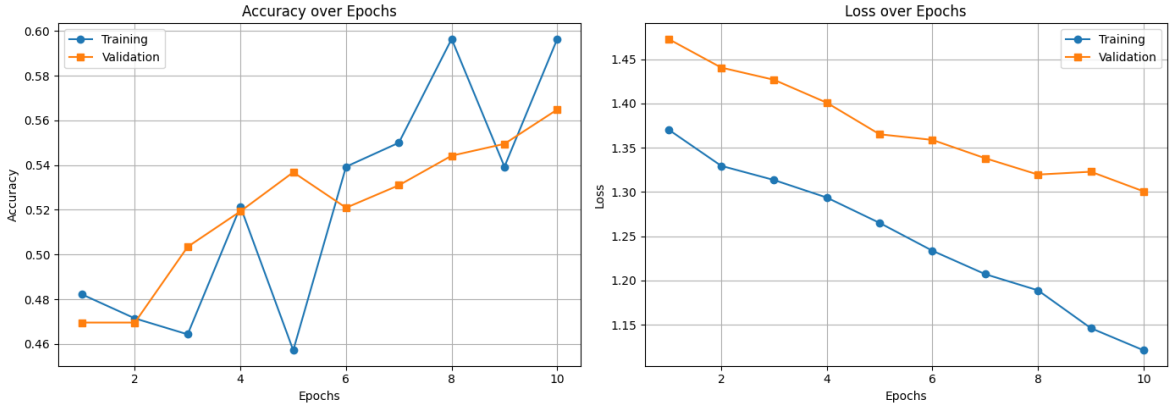
    # Training
    train_acc, train_loss = train(cassava_model,
                                  optimizer_cassava,
                                  criterion_cassava,
                                  cassava_train_dataloader,
                                  device,
                                  epoch,
                                  log_interval=10)
    train_accs.append(train_acc)
    train_losses.append(train_loss)

    # Evaluation
    eval_acc, eval_loss = evaluate(cassava_model,
                                   criterion_cassava,
                                   cassava_valid_dataloader,
                                   device)
    eval_accs.append(eval_acc)
    eval_losses.append(eval_loss)

    # Save best model
    if eval_loss < best_loss_eval:
        best_loss_eval = eval_loss
        torch.save(cassava_model.state_dict(),
                   os.path.join(save_model, "lenet_cassava_model.pt"))
```

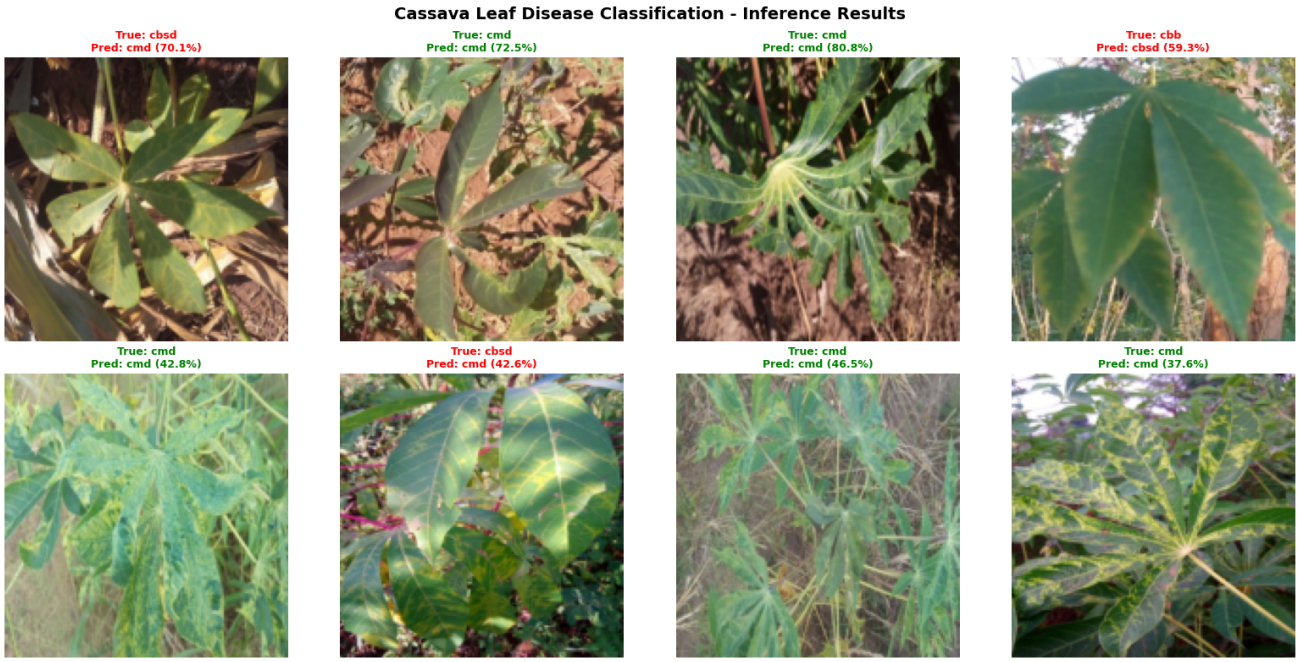
```
46
47      # Print results
48      print("-" * 80)
49      print(f"| End of epoch {epoch:3d} | Time: {time.time() - epoch_start_time:5.2f}s |
                                                "
50            f"Train Acc: {train_acc:8.3f} | Train Loss: {train_loss:8.3f} | "
51            f"Valid Acc: {eval_acc:8.3f} | Valid Loss: {eval_loss:8.3f} |")
52      print("-" * 80)
53
54  # Load best model
55  cassava_model.load_state_dict(
56      torch.load(os.path.join(save_model, "lenet_cassava_model.pt"),
57                 weights_only=True)
58  )
59  cassava_model.eval()
```

Trong bước này, ta khởi tạo mô hình `LeNetClassifier` cho bài toán phân loại 5 loại bệnh trên lá sắn, với số lớp đầu ra bằng số lớp trong `cassava_train_data` và số kênh đầu vào là 3 (ảnh màu RGB, kích thước 150×150). Hàm mất mát sử dụng là `CrossEntropyLoss`, còn tối ưu hoá sử dụng `Adam` với learning rate 2×10^{-4}.



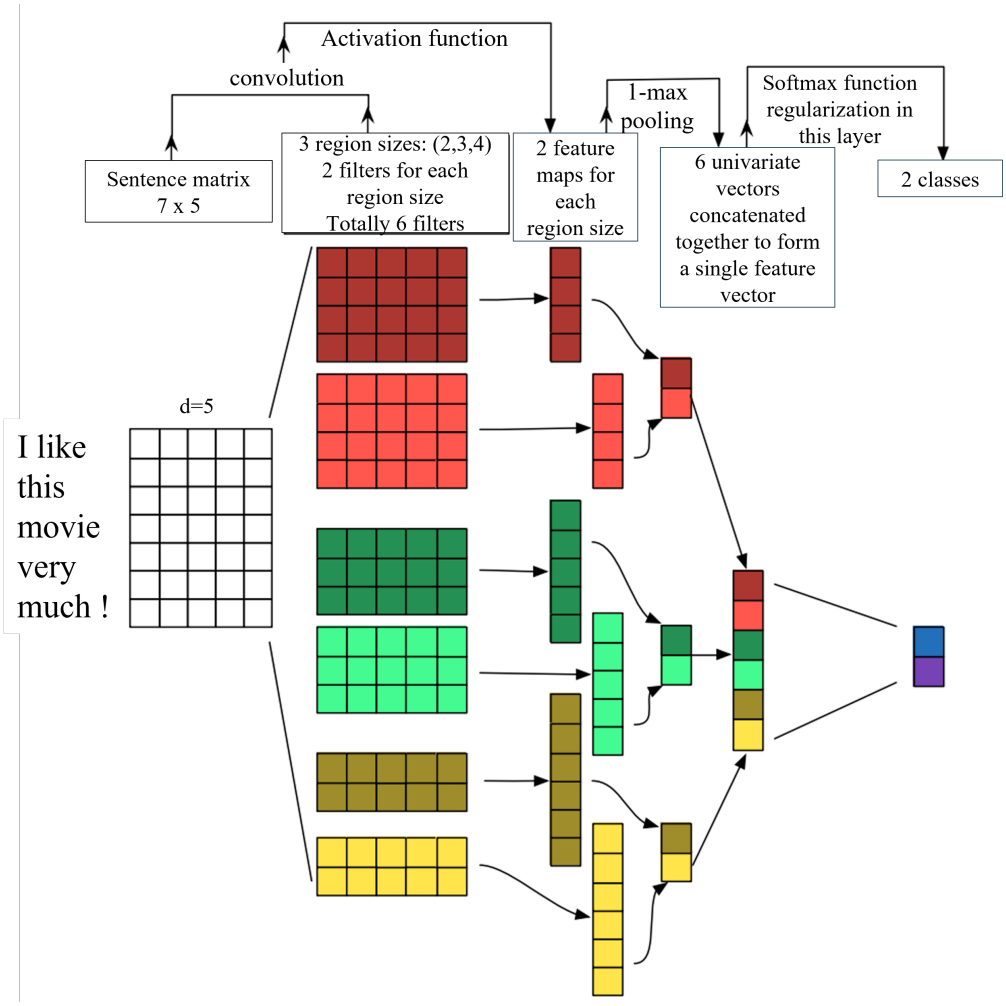
Hình 14: Biểu đồ loss của quá trình huấn luyện model.

Vòng lặp huấn luyện chạy trong 10 epoch. Ở mỗi epoch, mô hình được huấn luyện trên `cassava_train_datal` để thu được Train Acc và Train Loss, sau đó được đánh giá trên `cassava_valid_dataloader` để thu được Valid Acc và Valid Loss. Nếu Valid Loss tại một epoch tốt hơn giá trị tốt nhất hiện tại, trạng thái mô hình sẽ được lưu lại vào file `"lenet_cassava_model.pt"`. Kết thúc quá trình huấn luyện, ta nạp lại trọng số tốt nhất này và đưa mô hình về chế độ `eval()` để sẵn sàng cho bước đánh giá trên tập test và suy luận (inference) ở các phần tiếp theo.



Hình 15: Kết quả inference model trên tập test.

IV.3. Phân loại văn bản bằng mô hình TextCNN



Hình 16: Mô phỏng mô hình TextCNN đơn giản.

Hình trên minh họa kiến trúc TextCNN cho bài toán phân loại cảm xúc một câu tiếng Anh. Đầu vào là một câu "I like this movie very much!"được biểu diễn dưới dạng ma trận câu kích thước $7 \times d$, trong đó 7 là số từ trong câu và $d = 5$ là kích thước vector embedding của mỗi từ. Mỗi hàng trong ma trận tương ứng với một từ, còn mỗi cột là một chiều của embedding, tương tự như "ảnh 1 kênh"theo chiều dọc là các từ và chiều ngang là các đặc trưng embedding.

Trên ma trận câu này, mô hình áp dụng nhiều bộ lọc tích chập (convolution filters) với các kích thước vùng khác nhau để bắt các n-gram:

- Có 3 kích thước vùng (region sizes): 2, 3, 4 từ (tương ứng bi-gram, tri-gram, 4-gram).
- Với mỗi kích thước vùng, sử dụng 2 filters, tổng cộng 6 filters cho toàn bộ lớp convolution.
- Mỗi filter trượt theo chiều dọc (theo trục từ) của ma trận câu, sinh ra một feature map cho mỗi kích thước vùng.

Sau khi tích chập, đầu ra của từng filter được đưa qua một hàm kích hoạt phi tuyến (activation function, thường dùng ReLU) để giữ lại các phản hồi dương mạnh, tương tự như trong CNN xử lý ảnh.

Tiếp theo, mô hình áp dụng **1-max pooling** trên từng feature map: với mỗi filter, ta lấy giá trị lớn nhất trong toàn bộ chiều thời gian (theo trục từ). Như vậy, mỗi feature map được rút gọn thành đúng một giá trị vô hướng, biểu diễn "mức kích hoạt mạnh nhất"của pattern mà filter đó học được, bất kể pattern xuất hiện ở vị trí nào trong câu. Với 6 filters, ta thu được 6 giá trị vô hướng và nối (concatenate) chúng lại thành một vector đặc trưng duy nhất kích thước 6 chiều.

Cuối cùng, vector đặc trưng 6 chiều này được đưa vào một lớp dense (fully-connected) kèm theo hàm softmax để suy ra phân phối xác suất trên các lớp đầu ra (trong hình là 2 lớp, ví dụ: Positive vs Negative). Nhờ việc kết hợp nhiều kích thước vùng và cơ chế 1-max pooling, TextCNN có khả năng bắt được các cụm từ quan trọng (n-gram) ở bất kỳ vị trí nào trong câu và dùng chúng làm cơ sở cho quyết định phân loại văn bản.

Bước 1: Tiền xử lý văn bản, xây dựng từ vựng

```python
def preprocess_text(text):
    # Remove URLs
    url_pattern = re.compile(r"https?://\S+|www\.\S+")
    text = url_pattern.sub(r" ", text)

    # Remove HTML Tags
    html_pattern = re.compile(r"<[^<>]+>")
    text = html_pattern.sub(" ", text)

    # Remove punctuation and digits
    replace_chars = list(string.punctuation + string.digits)
    for char in replace_chars:
        text = text.replace(char, " ")

    # Remove emoji
    emoji_pattern = re.compile("["
        u"\U0001F600-\U0001F64F"
        u"\U0001F300-\U0001F5FF"
        u"\U0001F680-\U0001F6FF"
        u"\U0001F1E0-\U0001F1FF"
        u"\U0001F1F2-\U0001F1F4"
        u"\U0001F1E6-\U0001F1FF"
        u"\U0001F600-\U0001F64F"
        u"\U00002702-\U000027B0"
        u"\U000024C2-\U0001F251"
        u"\U0001f926-\U0001f937"
        u"\U0001F1F2"
        u"\U0001F1F4"
        u"\U0001F620"
        u"\u200d"
        u"\u2640-\u2642"
        "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r" ", text)
```

```
34
35      # Normalize whitespace and lowercase
36      text = " ".join(text.split()).lower()
37      return text
38
39  # Apply preprocessing
40  train_df["preprocess_sentence"] = train_df["sentence"].apply(preprocess_text)
41  valid_df["preprocess_sentence"] = valid_df["sentence"].apply(preprocess_text)
42  test_df["preprocess_sentence"]  = test_df["sentence"].apply(preprocess_text)
43
44  # Simple tokenizer: split by space
45  def simple_tokenizer(text):
46      return text.split()
47
48  # Build vocabulary
49  def build_vocab(texts, max_vocab_size=10000):
50      word_freq = Counter()
51      for text in texts:
52          tokens = simple_tokenizer(text)
53          word_freq.update(tokens)
54
55      # Get most common words
56      most_common = word_freq.most_common(max_vocab_size - 2)  # -2 for <pad> and <unk>
57
58      # Create word to index mapping
59      word2idx = {"<pad>": 0, "<unk>": 1}
60      for idx, (word, freq) in enumerate(most_common, start=2):
61          word2idx[word] = idx
62
63      return word2idx
64
65  # Encode function
66  def encode_text(text, word2idx):
67      tokens = simple_tokenizer(text)
68      return [word2idx.get(token, word2idx["<unk>"]) for token in tokens]
69
70  # Build vocabulary from training data
71  word2idx = build_vocab(train_df["preprocess_sentence"], max_vocab_size=10000)
72  vocab_size = len(word2idx)
73
74  # Custom Dataset
75  class TextDataset(Dataset):
76      def __init__(self, df, word2idx):
77          self.texts = df["preprocess_sentence"].values
78          self.labels = df["label"].values
79          self.word2idx = word2idx
80
81      def __len__(self):
82          return len(self.texts)
83
84      def __getitem__(self, idx):
85          encoded = encode_text(self.texts[idx], self.word2idx)
86          return torch.tensor(encoded, dtype=torch.long), \
87                 torch.tensor(self.labels[idx], dtype=torch.long)
```

```
88
89 # Collate function for batching
90 def collate_batch(batch):
91     encoded_sentences, labels = [], []
92     for encoded, label in batch:
93         encoded_sentences.append(encoded)
94         labels.append(label)
95
96     # Pad sequences
97     encoded_sentences = pad_sequence(encoded_sentences,
98                                      padding_value=word2idx["<pad>"])
99     labels = torch.tensor(labels, dtype=torch.long)
100
101    return encoded_sentences, labels
102
103 # Create datasets
104 text_train_dataset = TextDataset(train_df, word2idx)
105 text_valid_dataset = TextDataset(valid_df, word2idx)
106 text_test_dataset  = TextDataset(test_df, word2idx)
107
108 # Create dataloaders
109 batch_size = 128
110 text_train_dataloader = DataLoader(text_train_dataset,
111                                    batch_size=batch_size,
112                                    shuffle=True,
113                                    collate_fn=collate_batch)
114 text_valid_dataloader = DataLoader(text_valid_dataset,
115                                    batch_size=batch_size,
116                                    shuffle=False,
117                                    collate_fn=collate_batch)
118 text_test_dataloader  = DataLoader(text_test_dataset,
119                                    batch_size=batch_size,
120                                    shuffle=False,
121                                    collate_fn=collate_batch)
```

Trong bước này, dữ liệu văn bản được tiền xử lý và chuyển sang dạng số trước khi đưa vào mô hình TextCNN. Hàm `preprocess_text` lần lượt loại bỏ URL, thẻ HTML, dấu câu, chữ số, emoji, chuẩn hóa khoảng trắng và chuyển toàn bộ câu về chữ thường, tạo thành cột `"preprocess_sentence"` cho train/validation/test. Sau đó, ta dùng tokenizer đơn giản (tách theo dấu cách) và xây dựng một từ vựng (`word2idx`) dựa trên tần suất xuất hiện của từ trong tập train, trong đó hai ký hiệu đặc biệt `"<pad>"` và `"<unk>"` được gán cho các vị trí 0 và 1.

Mỗi câu được mã hóa thành một dãy chỉ số bằng hàm `encode_text`. Lớp `TextDataset` lưu trữ câu đã tiền xử lý và nhãn tương ứng, đồng thời khi truy cập từng phần tử sẽ trả về cặp `(sequence_ids, label)` dưới dạng tensor. Vì độ dài câu trong batch có thể khác nhau, hàm `collate_batch` được dùng để padding các chuỗi về cùng độ dài bằng `pad_sequence`, sử dụng chỉ số `"<pad>"` làm giá trị đệm. Cuối cùng, các `DataLoader` cho train/validation/test được tạo ra với `batch_size = 128`, trong đó tập train được shuffle để mô hình học ổn định hơn, còn tập validation và test được giữ nguyên thứ tự để phục vụ đánh giá.

Bước 2: Khai báo kiến trúc TextCNN

Code Exercise

```python
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, kernel_sizes,
                 num_filters, num_classes):
        super(TextCNN, self).__init__()

        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.kernel_sizes = kernel_sizes
        self.num_filters = num_filters
        self.num_classes = num_classes

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.conv = nn.ModuleList([
            nn.Conv1d(in_channels=embedding_dim,
                      out_channels=num_filters,
                      kernel_size=k,
                      stride=1)
            for k in kernel_sizes
        ])
        self.fc = nn.Linear(len(kernel_sizes) * num_filters, num_classes)

    def forward(self, x):
        # x shape: (seq_len, batch_size)
        x = ***Your code here***  # (batch_size, embedding_dim, seq_len)
        x = [F.relu(conv(x)) for conv in self.conv]
        x = [F.max_pool1d(c, c.size(-1)).squeeze(dim=-1) for c in x]
        x = torch.cat(x, dim=1)
        x = ***Your code here***
        return x
```

Trong bước này, bạn cần **hoàn thiện lớp TextCNN** bằng cách điền vào hai vị trí `***Your code here***` trong hàm `forward` sao cho mô hình hiện thực đúng kiến trúc đã mô tả ở phần trước.

Cụ thể:

- Đầu vào `x` có shape (seq_len, batch_size); ở bước đầu tiên trong `forward`, bạn phải đưa `x` qua lớp `embedding` rồi hoán vị các chiều để thu được tensor có dạng (batch_size, embedding_dim, seq_l phù hợp với yêu cầu của `Conv1d`.
- Sau đó, tensor này lần lượt đi qua các lớp `conv` với các kích thước kernel khác nhau, được kích hoạt bằng ReLU, rồi áp dụng `max_pool1d` trên toàn bộ chiều thời gian để thu được một vector cho mỗi filter; các vector này được nối lại bằng `torch.cat` để tạo thành một feature vector chung cho cả câu.
- Ở vị trí `***Your code here***` cuối cùng, bạn cần đưa feature vector này qua lớp `fc` để ánh xạ sang không gian `num_classes` và thu được logits phục vụ cho bài toán phân loại văn bản.

Bước 3: Khởi tạo và huấn luyện mô hình TextCNN

```python
# Model setup (vocab_size đã được tạo ở cell trước)
embedding_dim = 100

text_model = TextCNN(
    vocab_size=vocab_size,
    embedding_dim=embedding_dim,
    kernel_sizes=[3, 4, 5],
    num_filters=100,
    num_classes=2
).to(device)

print(f"Text Model initialized on {device}")
print(f"Vocab size: {vocab_size}, Embedding dim: {embedding_dim}")
print(f"Total parameters: {sum(p.numel() for p in text_model.parameters()):,}")

criterion_text = nn.CrossEntropyLoss()
optimizer_text = optim.Adam(text_model.parameters())

# Training loop
num_epochs = 4
save_model = "./model"
os.makedirs(save_model, exist_ok=True)

train_accs, train_losses = [], []
eval_accs, eval_losses = [], []
best_loss_eval = 100

for epoch in range(1, num_epochs + 1):
    epoch_start_time = time.time()

    # Training
    train_acc, train_loss = train(text_model,
                                  optimizer_text,
                                  criterion_text,
                                  text_train_dataloader,
                                  device,
                                  epoch,
                                  log_interval=50)
    train_accs.append(train_acc)
    train_losses.append(train_loss)

    # Evaluation
    eval_acc, eval_loss = evaluate(text_model,
                                   criterion_text,
                                   text_valid_dataloader,
                                   device)
    eval_accs.append(eval_acc)
    eval_losses.append(eval_loss)

    # Save best model
    if eval_loss < best_loss_eval:
```

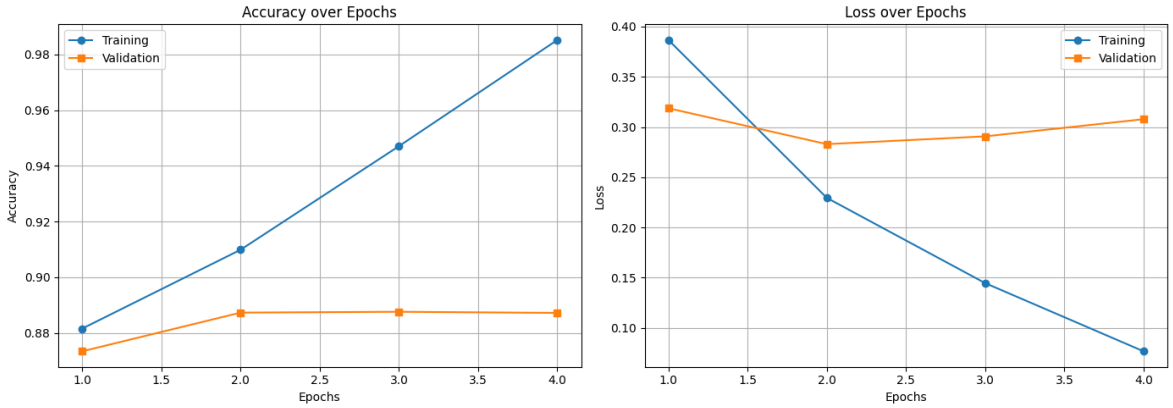
```
        best_loss_eval = eval_loss
        torch.save(text_model.state_dict(),
                   os.path.join(save_model, "text_cnn_model.pt"))

    # Print results
    print("-" * 80)
    print(f"| End of epoch {epoch:3d} | Time: {time.time() - epoch_start_time:5.2f}s |
                                                "
          f"Train Acc: {train_acc:8.3f} | Train Loss: {train_loss:8.3f} | "
          f"Valid Acc: {eval_acc:8.3f} | Valid Loss: {eval_loss:8.3f} |")
    print("-" * 80)

# Load best model
text_model.load_state_dict(
    torch.load(os.path.join(save_model, "text_cnn_model.pt"),
               weights_only=True)
)
text_model.eval()
```

Trong bước này, ta khởi tạo mô hình `TextCNN` với kích thước embedding 100, ba kích thước kernel $[3, 4, 5]$, mỗi loại 100 filters và số lớp đầu ra bằng 2 (ví dụ: Positive/Negative). Mô hình được đưa lên `device` (CPU hoặc GPU), kèm theo thông tin về kích thước từ vựng, số chiều embedding và tổng số tham số. Hàm mất mát sử dụng là `CrossEntropyLoss`, còn optimizer là `Adam` với thiết lập mặc định.

Vòng lặp huấn luyện chạy trong 4 epoch: ở mỗi epoch, mô hình được train trên `text_train_dataloader` để thu được Train Acc và Train Loss, sau đó đánh giá trên `text_valid_dataloader` để lấy Valid Acc và Valid Loss. Nếu Valid Loss tại epoch hiện tại thấp hơn giá trị tốt nhất trước đó, trạng thái mô hình sẽ được lưu vào file `"text_cnn_model.pt"`. Cuối cùng, ta nạp lại checkpoint tốt nhất này và đưa mô hình về chế độ `eval()` để phục vụ cho bước đánh giá chi tiết và suy luận trên dữ liệu mới.



Hình 17: Biểu đồ loss của quá trình huấn luyện model.

Cuối cùng ta inference trên một số ví dụ bằng mô hình TextCNN sau khi đã huấn luyện. Mỗi mẫu bao gồm câu đầu vào, nhãn dự đoán và độ tự tin tương ứng. Kết quả cho thấy mô hình có khả năng phân biệt tốt giữa hai lớp cảm xúc, thể hiện qua các dự đoán nhất quán và độ tin cậy cao trên nhiều câu khác nhau.

```
Sentence: Sản phẩm rất tốt, tôi rất hài lòng!
Sentiment: Positive, Confidence: 89.48%
--------------------------------------------------
Sentence: Giao hàng nhanh, đóng gói cẩn thận.
Sentiment: Positive, Confidence: 84.02%
--------------------------------------------------
Sentence: Chất lượng tệ, không đáng tiền.
Sentiment: Negative, Confidence: 99.18%
--------------------------------------------------
Sentence: Sản phẩm bị hỏng, dịch vụ hỗ trợ rất chậm.
Sentiment: Negative, Confidence: 84.63%
--------------------------------------------------
Sentence: Màu sắc khác hoàn toàn so với hình, tôi rất thất vọng.
Sentiment: Negative, Confidence: 77.75%
--------------------------------------------------
```

Hình 18: Vài mẫu kết quả suy luận của model phân loại văn bản.

V. Câu hỏi trắc nghiệm

1. (Lý thuyết) Tại sao việc dùng MLP fully-connected trực tiếp trên ảnh lớn (ví dụ 1000×1000 pixels) thường không khả thi trong thực tế?

   (a) Vì MLP không thể xử lý dữ liệu dạng ma trận, chỉ xử lý được vector 1 chiều.

   (b) Vì số tham số tăng cực lớn, dễ overfitting, đòi hỏi rất nhiều dữ liệu, bộ nhớ và tài nguyên tính toán.

   (c) Vì MLP không thể dùng hàm kích hoạt phi tuyến nên không học được feature phức tạp.

   (d) Vì MLP bắt buộc phải dùng ảnh màu, không dùng được ảnh xám.

2. (Lý thuyết) Hai tính chất quan trọng nào của ảnh được CNN khai thác để thiết kế kiến trúc hiệu quả?

   (a) Feature localization và feature independence of location.

   (b) Feature independence of channel và feature redundancy.

   (c) Temporal locality và causal dependence.

   (d) Spatial stationarity và time invariance theo trục thời gian.

3. (Lý thuyết) Phát biểu nào mô tả đúng ba khái niệm chính trong CNN: Convolution Layer, Pooling Layer và Fully-Connected Layer?

   (a) Convolution Layer giảm số kênh, Pooling Layer tăng độ phân giải, Fully-Connected Layer chỉ dùng để nén ảnh.

   (b) Convolution Layer trích xuất đặc trưng cục bộ, Pooling Layer giảm kích thước không gian và tăng tính bất biến, Fully-Connected Layer kết hợp các đặc trưng để ra quyết định phân loại.

   (c) Cả ba lớp đều chỉ dùng để regularization, không ảnh hưởng đến biểu diễn đặc trưng.

   (d) Convolution Layer chỉ thay màu pixel, Pooling Layer chỉ xoay ảnh, Fully-Connected Layer chỉ làm chuẩn hoá.

4. (Tính toán) Convolution với padding. Cho một ảnh đầu vào có chiều cao $H = 28$. Ta áp dụng một convolution 2D với kernel cao $K_h = 3$, stride $S_h = 1$ và padding theo chiều cao $P_h = 1$. Kích thước chiều cao đầu ra được tính theo công thức:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1.$$

   Giá trị H_{out} là:

   (a) $H_{\text{out}} = 26$

   (b) $H_{\text{out}} = 27$

   (c) $H_{\text{out}} = 28$

(d) $H_{\text{out}} = 29$

5. (Tính toán) Convolution với stride lớn hơn 1. Cho một ảnh đầu vào có chiều rộng $W = 32$. Ta áp dụng một convolution 2D với kernel rộng $K_w = 5$, padding theo chiều rộng $P_w = 0$ và stride $S_w = 2$. Kích thước chiều rộng đầu ra được tính theo công thức:

$$W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Giá trị W_{out} là:

(a) $W_{\text{out}} = 13$

(b) $W_{\text{out}} = 14$

(c) $W_{\text{out}} = 15$

(d) $W_{\text{out}} = 16$

6. (Tính toán) Shape đầu ra của Convolution Layer 2D tổng quát. Một feature map đầu vào có kích thước $(C_{\text{in}}, H, W) = (3, 64, 64)$. Ta sử dụng một convolution layer với số filter $C_{\text{out}} = 16$, kernel $K_h = K_w = 3$, padding $P_h = P_w = 1$, stride $S_h = S_w = 1$. Kích thước đầu ra được tính bởi:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Shape đầu ra $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ là:

(a) $(3, 64, 64)$

(b) $(16, 64, 64)$

(c) $(16, 62, 62)$

(d) $(32, 64, 64)$

7. (Tính toán) Convolution kết hợp kernel, padding và stride. Cho ảnh đầu vào kích thước $(C_{\text{in}}, H, W) = (1, 28, 28)$. Ta dùng một convolution với:

$$K_h = K_w = 5, \quad P_h = P_w = 2, \quad S_h = S_w = 2, \quad C_{\text{out}} = 8.$$

Kích thước đầu ra được tính bởi:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Shape đầu ra $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ là:

(a) $(8, 12, 12)$

(b) $(8, 14, 14)$

(c) $(16, 14, 14)$

(d) $(8, 28, 28)$

8. (Tính toán) Max-pooling 2D thường gặp (kernel 2×2, stride 2). Một feature map có kích thước $(C, H, W) = (6, 26, 26)$ được đưa qua max-pooling 2D với kernel $(K_h, K_w) = (2, 2)$ và stride $(S_h, S_w) = (2, 2)$, không padding. Kích thước đầu ra được tính bởi:

$$H_{\text{out}} = \left\lfloor \frac{H - K_h}{S_h} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W - K_w}{S_w} \right\rfloor + 1.$$

Shape đầu ra $(C, H_{\text{out}}, W_{\text{out}})$ là:

(a) $(6, 13, 13)$
(b) $(6, 12, 12)$
(c) $(3, 13, 13)$
(d) $(6, 24, 24)$

9. (Tính toán) So sánh pooling 1D và 2D. Xét một tín hiệu 1D có độ dài L, được đưa qua max-pooling 1D với kernel K và stride S, không padding. Kích thước đầu ra được tính bởi:

$$L_{\text{out}} = \left\lfloor \frac{L - K}{S} \right\rfloor + 1.$$

Với $L = 10$, $K = 2$, $S = 2$, ta có:

$$L_{\text{out}} = \left\lfloor \frac{10 - 2}{2} \right\rfloor + 1.$$

Giá trị L_{out} là:

(a) $L_{\text{out}} = 4$
(b) $L_{\text{out}} = 5$
(c) $L_{\text{out}} = 6$
(d) $L_{\text{out}} = 8$

10. (Tính toán) Luồng shape qua một pipeline CNN đơn giản. Xét mô hình CNN đơn giản cho MNIST với pipeline:

$$(1, 28, 28) \ \rightarrow \ \text{Conv1} : C_{\text{out}} = 6, K = 5, P = 0, S = 1 \ \rightarrow \ \text{MaxPool1} : K = 2, S = 2$$
$$\rightarrow \ \text{Conv2} : C_{\text{out}} = 16, K = 5, P = 0, S = 1 \ \rightarrow \ \text{MaxPool2} : K = 2, S = 2 \ \rightarrow \ \text{Flatten}.$$

Với mỗi convolution và pooling, kích thước chiều cao/chiều rộng được tính theo:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \qquad H_{\text{out}}^{\text{pool}} = \left\lfloor \frac{H - K}{S} \right\rfloor + 1$$

(tương tự cho W). Sau toàn bộ hai Convolution + Pooling, đầu ra có shape $(C_{\text{final}}, H_{\text{final}}, W_{\text{final}})$ và được Flatten thành vector có độ dài:

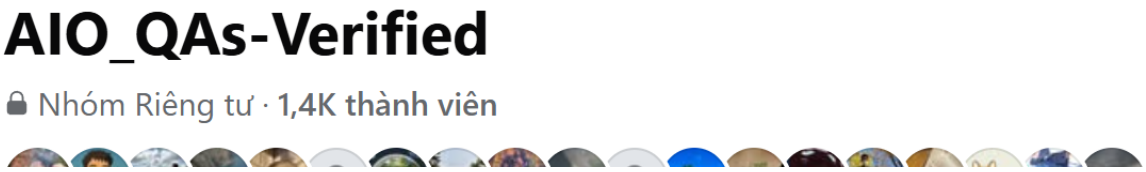
$$D = C_{\text{final}} \cdot H_{\text{final}} \cdot W_{\text{final}}.$$

Giá trị D (số chiều đầu vào của fully-connected layer đầu tiên) là:

(a) $D = 64$
(b) $D = 128$
(c) $D = 256$
(d) $D = 512$

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải tại đây.
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (Lưu ý: Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.



Hình 19: Hình ảnh group facebook AIO Q&A

4. **Rubric:**

Phần	Kiến Thức	Đánh Giá
1	- Nắm vững kiến thức về bài toán phân loại hình ảnh. - Kiến trúc mạng LeNet. - Các thành phần quan trọng xây dựng mô hình LeNet như: Lớp tích chập, lớp pooling, lớp linear.	- Xây dựng mô hình phân loại hình ảnh sử dụng mạng LeNet trên bộ dữ liệu MNIST và Cassava Leaf Disease.
2	- Hiểu về mô hình phân loại văn bản sử dụng TextCNN. - Các lớp xử lý cho văn bản như Embedding thông qua thư viện `torchtext`.	- Xây dựng mô hình phân loại văn bản sử dụng TextCNN. - Bộ dữ liệu phân loại văn bản NTC-SCV.